

PBCST404

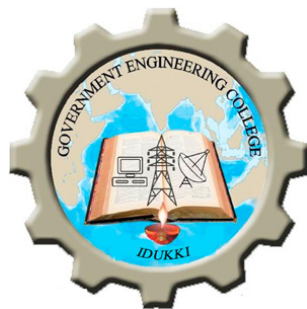
COMPUTER ORGANIZATION AND ARCHITECTURE

**ARCHITECTURE AND SIMULATION OF DMA
CONTROLLER**

Submitted by

ADIL ZAMAN V

PROJECT REPORT



Department of Computer Science & Engineering

GOVERNMENT ENGINEERING COLLEGE, IDUKKI

MARCH - 2026

ABSTRACT

This project presents the design and implementation of a Direct Memory Access (DMA) Controller that enables high-speed data transfer between memory and I/O devices with minimal CPU involvement, thereby improving overall system efficiency. The DMA controller supports multiple transfer modes, including Cycle Stealing Mode, where the DMA temporarily takes control of the system bus for single data transfers allowing the CPU to continue execution with slight delays, and Burst Mode, where the DMA transfers a block of data continuously by fully occupying the bus, increasing throughput while temporarily halting CPU operations. The design incorporates essential bus arbitration signals such as Bus Request (BR) and Bus Grant (BG) to coordinate access between the CPU and DMA controller, ensuring proper control of the system bus during data transfers. The system is implemented and simulated using digital design tools, demonstrating the functionality of different DMA modes and highlighting reduced CPU workload, efficient bus utilization, and enhanced data transfer performance, providing a clear understanding of DMA architecture and its role in modern computer systems.

Contents

Abstract	i
1 Introduction	1
2 Basics of Logisim	3
2.1 Introduction to Logisim	3
2.2 Logisim Environment and Interface	3
2.3 Basic Circuit Design in Logisim	4
2.4 Multi-bit Circuits and Use of Buses	4
2.5 Simulation Techniques in Logisim	5
2.6 Flip-Flops and Sequential Logic	5
2.7 Conclusion	6
3 Direct Memory Access	7
3.1 The Necessity of DMA	7
3.2 Programmed I/O vs. DMA	7
3.3 DMA Operational Circle	7
3.4 Modes of Operation	8
4 Logisim Circuit Design and Implementation	9
4.1 Circuit Components and Logic	9
4.2 The DMA Handshake Protocol	9
4.3 Implementation of DMA Modes	9
4.3.1 Burst Mode Implementation	9
4.3.2 Cycle Stealing Mode Implementation	10
4.4 Comparison of Control Signals	11
5 Code Explanation	12
5.1 Program 1: GUI-Based DMA Simulation	12
5.1.1 IO Device Implementation	12
5.1.2 DMA Controller Implementation	12
5.1.3 GUI Visualization	13
5.1.4 Performance Graph	13
5.1.5 Output (Program 1)	14
5.2 Program 2: Thread-Based DMA Simulation	15
5.2.1 Main Memory Implementation	15
5.2.2 Data Transfer Logic	15
5.2.3 DMA Configuration	16
5.2.4 CPU Parallel Execution	16
5.2.5 Output (Program 2)	16
5.3 Summary	17
6 Objectives	18

7	Experiment and Result Analysis	19
7.1	Experimental Setup	19
7.2	Result Analysis: Logisim Hardware Logic	19
7.3	Result Analysis: Python Performance Data	20
7.4	Analysis of the 1.54s Gap	20
7.5	Conclusion of the Experiment	20
8	Conclusion	21

List of Figures

2.1	Logisim User Interface	4
2.2	XNOR Gate Implementation using Basic Gates	4
2.3	Multi-bit Equality Checking Circuit	5
2.4	D Flip-Flop in Logisim	6
4.1	Burst Mode of DMA Circuit	10
4.2	Cycle Stealing of DMA Circuit	11
5.1	DMA GUI Simulation	14
5.2	Cycle Stealing	14
5.3	Burst mode	15
5.4	Output of Program 2	16

List of Tables

3.1	Comparison of Programmed I/O and DMA	7
4.1	Comparison of Control Signals in DMA Modes	11
7.1	Performance Comparison of DMA Modes	19

1. Introduction

In modern computer systems, efficient data transfer between memory and input/output (I/O) devices is essential for achieving high system performance. Traditionally, the Central Processing Unit (CPU) is responsible for managing all data transfers, including those between peripheral devices and main memory. However, this approach can significantly burden the CPU, as it must continuously monitor and control every transfer operation. This leads to inefficient utilization of processing resources, especially in systems where large volumes of data need to be moved frequently.

To overcome these limitations, Direct Memory Access (DMA) was introduced as a specialized data transfer mechanism. A DMA controller enables data transfer between memory and I/O devices without continuous CPU intervention. By temporarily taking control of the system bus, the DMA controller allows data to be transferred directly, freeing the CPU to perform other tasks. This significantly improves overall system efficiency, reduces latency, and enhances throughput.

The concept of DMA is particularly important in high-speed systems where real-time data processing is required. Applications such as disk operations, multimedia streaming, and network communication rely heavily on efficient data transfer mechanisms. Without DMA, the CPU would become a bottleneck, limiting system performance. Therefore, understanding and implementing DMA controllers is a crucial aspect of computer architecture and digital system design.

This project focuses on the design and simulation of a DMA Controller using digital logic principles and simulation tools. The implementation is carried out using Logisim for hardware-level visualization and Python for behavioral simulation and analysis. The Logisim-based design provides a clear understanding of how hardware components such as registers, counters, control logic, and bus arbitration units interact to perform DMA operations. On the other hand, the Python implementation helps simulate the logical flow of data transfer, allowing observation of system behavior, timing, and performance characteristics.

The DMA controller designed in this project operates in cycle stealing mode, where the controller temporarily suspends CPU operations to transfer a single data word. After the transfer, control is returned to the CPU. This approach provides a balance between CPU operation and data transfer efficiency, ensuring that neither process is completely blocked for extended periods. The implementation demonstrates how the DMA controller requests bus access, transfers data, updates memory addresses, and signals completion of the transfer.

Another important aspect of this project is the visualization and analysis of the DMA process. The Python-based simulation includes graphical representations and user interaction, enabling better understanding of how data flows through the system. By combining hardware simulation with software modeling, the project provides a comprehensive view of DMA oper-

ation from both structural and functional perspectives. The significance of this project lies not only in understanding DMA as a theoretical concept but also in gaining practical experience in designing and simulating digital systems. It bridges the gap between hardware-level design and software-level implementation, making it easier to grasp complex system interactions. Furthermore, it highlights the importance of optimizing resource usage in computer systems, a key principle in modern computing.

In summary, this report presents the design, implementation, and analysis of a DMA controller using both Logisim and Python. It explores the working principles of DMA, demonstrates its advantages over CPU-controlled data transfer, and provides insights into its real-world applications. The project serves as a valuable learning tool for understanding advanced concepts in computer organization and digital system design.

2. Basics of Logisim

2.1 Introduction to Logisim

Logisim is a graphical digital logic design and simulation tool widely used for educational purposes in the field of computer organization and digital electronics. It was developed to help students understand the fundamental concepts of logic circuits through visual representation and real-time simulation. Unlike traditional theoretical approaches, Logisim allows users to construct circuits using basic components such as logic gates, wires, multiplexers, and memory elements. The behavior of these circuits can be observed instantly, which makes it easier to understand how digital systems operate internally. Due to its simplicity and effectiveness, Logisim is commonly used for designing combinational and sequential circuits, including complex systems such as arithmetic units and controllers like DMA.

2.2 Logisim Environment and Interface

When Logisim is launched, the user is presented with an interactive graphical interface that consists of several important sections. These sections collectively provide all the tools required to design, modify, and simulate digital circuits.

The main components of the Logisim environment are described below:

- **Toolbar:** The toolbar contains frequently used tools such as the poke tool, input pins, output pins, and wiring tools. The poke tool is particularly useful as it allows users to change input values dynamically during simulation.
- **Explorer Pane:** This section displays all available components that can be used in circuit design. These include logic gates (AND, OR, NOT, XOR), wiring components, arithmetic units, and memory elements. Users can select components from this pane and place them onto the canvas.
- **Attribute Table:** The attribute table provides configurable properties for the selected component. For example, users can change the number of inputs of a gate, adjust bit width, or modify behavior parameters.
- **Canvas:** The canvas is the workspace where circuits are created. Components are placed here and connected using wires. The simulation runs directly on the canvas, allowing real-time observation of outputs.

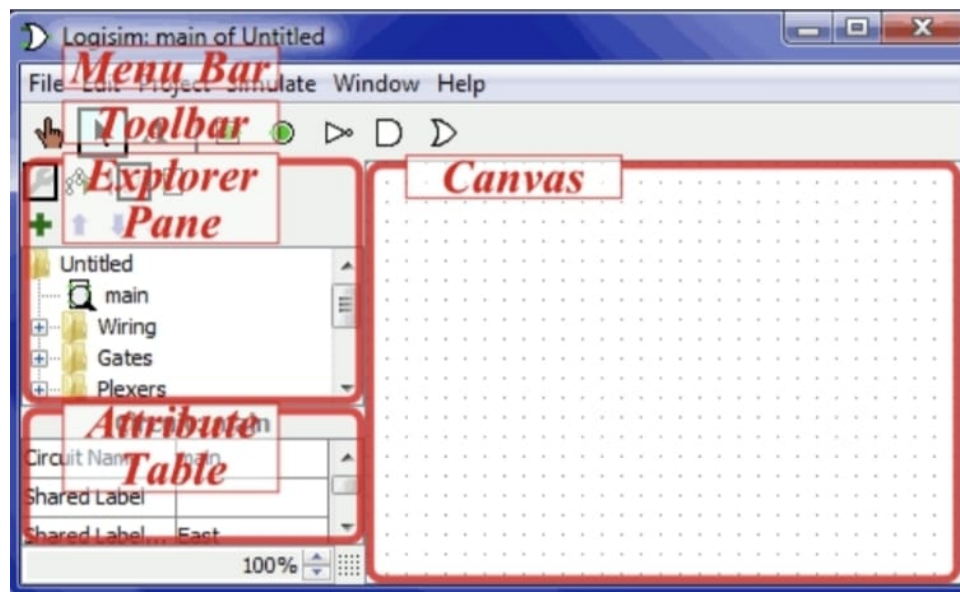


Figure 2.1: Logisim User Interface

2.3 Basic Circuit Design in Logisim

Logisim enables users to design circuits by placing components and connecting them logically. A simple example is the construction of an XNOR gate using basic logic gates such as AND, OR, and NOT. In this design, the XNOR function outputs a logic '1' when both inputs are equal and '0' otherwise. This demonstrates how complex logic functions can be built using simpler components. Such designs help in understanding the building blocks of digital systems.

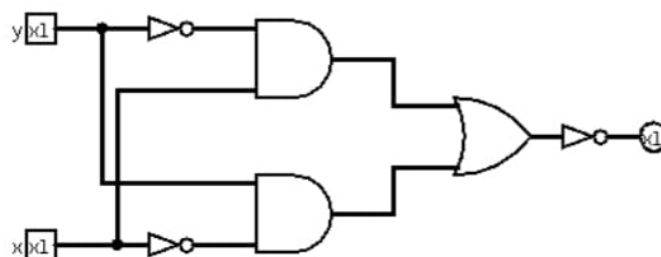


Figure 2.2: XNOR Gate Implementation using Basic Gates

2.4 Multi-bit Circuits and Use of Buses

In practical digital systems, data is often represented using multiple bits. Logisim supports multi-bit operations using buses and splitters. A bus is a collection of multiple wires represented as a single thick line. It is used to carry multi-bit data efficiently. Splitters are used to divide a bus into individual bits so that each bit can be processed separately. Using these components, complex circuits such as multi-bit comparators and arithmetic units can be designed. This feature is particularly important when implementing real-world digital systems.

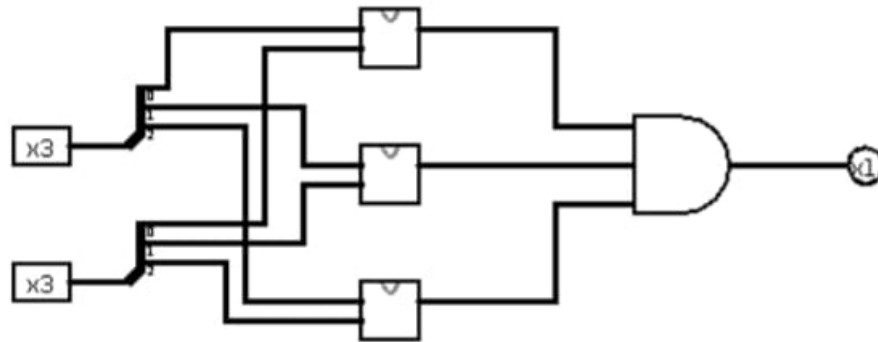


Figure 2.3: Multi-bit Equality Checking Circuit

2.5 Simulation Techniques in Logisim

One of the key advantages of Logisim is its powerful simulation capabilities. It allows users to test and verify circuit behavior before physical implementation.

- **Simulate Enabled:** This mode allows the circuit to run continuously and update outputs in real time based on input changes.
- **Step Simulation:** In this mode, the circuit is executed step by step. This is useful for analyzing signal propagation through each component.
- **Tick Simulation:** This mode is used for clock-based circuits. It allows the user to control clock pulses manually or automatically.

These simulation methods are essential for debugging and understanding both combinational and sequential circuits.

2.6 Flip-Flops and Sequential Logic

Sequential circuits differ from combinational circuits in that they have memory. Flip-flops are the fundamental building blocks of such circuits.

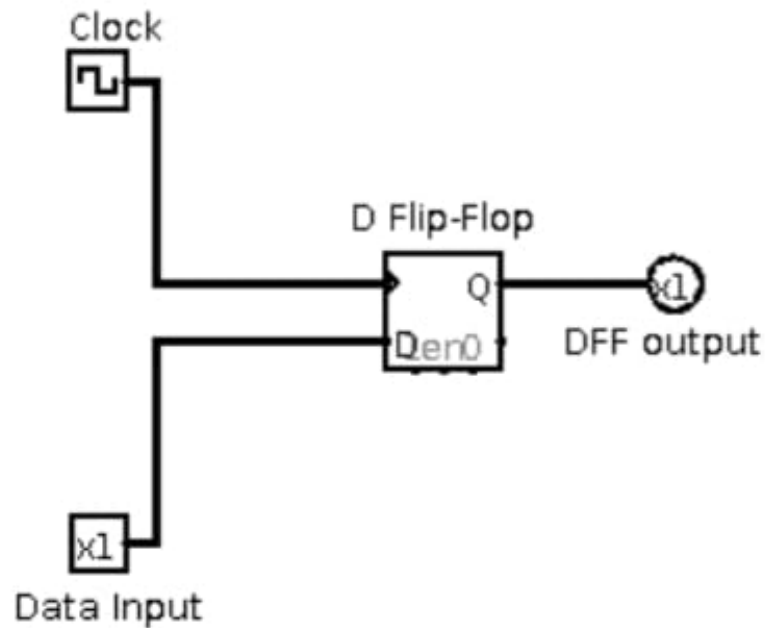


Figure 2.4: D Flip-Flop in Logisim

A D flip-flop stores a single bit of data and updates its output based on the clock signal. The important inputs of a D flip-flop include:

- **D (Data Input):** The value to be stored
- **Clock:** Controls when the value is stored
- **Q (Output):** Stored value output

The output changes only at the rising edge of the clock signal, making flip-flops suitable for designing registers, counters, and memory units.

2.7 Conclusion

Logisim provides a simple yet powerful platform for designing and simulating digital circuits. Its graphical interface, combined with real-time simulation, makes it an ideal tool for understanding fundamental concepts in computer organization. By using Logisim, complex systems such as DMA controllers can be designed, tested, and analyzed effectively before hardware implementation.

3. Direct Memory Access

3.1 The Necessity of DMA

In a traditional computer architecture, the CPU acts as the "middleman" for all data movement. If a hard drive needs to send a block of data to the System RAM, the CPU must read each byte from the I/O port and then write it into a memory address. This method, known as Programmed I/O (PIO), is highly inefficient because:

- The CPU is "trapped" in a fetch-execute loop just to move data.
- The high-speed System Bus is underutilized.
- System performance drops during heavy I/O tasks (like loading a game or saving a large file).

Direct Memory Access (DMA) solves this by allowing a specialized hardware module—the DMA Controller (DMAC)—to take over the bus and move data directly between I/O and Memory.

3.2 Programmed I/O vs. DMA

To understand the impact of your project, it is helpful to compare how the system behaves with and without a DMA controller:

Table 3.1: Comparison of Programmed I/O and DMA

Feature	Programmed I/O (PIO)	Direct Memory Access (DMA)
Data Transfer Control	CPU manages all transfers	DMA Controller handles transfers
CPU Utilization	High (CPU waits for I/O)	Low (CPU performs other tasks)
Transfer Speed	Slow	Fast
Bus Utilization	Inefficient	Highly Efficient
System Performance	Drops during heavy I/O	Maintained during heavy I/O

3.3 DMA Operational Circle

A DMA operation follows a strict four-step lifecycle. This cycle ensures that the CPU and the DMA Controller do not try to use the system bus at the same time, which would cause a "bus contention" error.

1. **Initialization:** The CPU writes to the DMA registers to define the Source Address, Destination Address, and Transfer Count.
2. **Request:** The DMA Controller senses that an I/O device is ready and sends a Bus Request to the CPU.
3. **Transfer:** Once the CPU sends a Bus Grant, the DMA Controller takes control of the Address and Data buses to move the information.
4. **Completion:** Once the transfer count reaches zero, the DMA Controller releases the bus and sends an Interrupt to the CPU to signal that the task is finished.

3.4 Modes of Operation

There are different ways a DMA can "share" the bus with the CPU. In your project, you have implemented the two most common methods:

- **Burst Mode:** The DMA takes the bus and doesn't let go until the entire block of data is moved. This is the "sprint" method—fastest for the data, but the CPU is completely paused.
- **Cycle Stealing Mode:** The DMA takes the bus for only one clock cycle at a time. It "steals" a moment, moves one byte, and gives the bus back to the CPU. This is the "marathon" method—it takes longer to finish, but the CPU stays responsive.

4. Logisim Circuit Design and Implementation

This section detailing the hardware implementation of the DMA Controller using Logisim. The design focuses on the "Bus Handshake" protocol, which ensures the CPU and DMA Controller never attempt to drive the system bus simultaneously.

4.1 Circuit Components and Logic

In the Logisim diagrams, several key components work together to facilitate the transfer:

- **State Flip-Flops:** Used to "latch" the start signal and maintain the Bus Request status until the transfer is complete.
- **Address Counter:** Acts as the DMA Address Register (DAR). It increments with every clock pulse to select the next memory location.
- **AND Gate Logic:** In the Cycle Stealing model, this gate acts as a "Gatekeeper," ensuring the counter only increments when (Clock AND Bus_Grant) are both High.
- **RAM Block:** Represents the system memory where data is stored or retrieved.

4.2 The DMA Handshake Protocol

The core of the hardware logic is the interaction between the CPU and the DMA Controller via two primary control signals:

- **Bus Request (BR):** An output from the DMA Controller to the CPU, signaling that it needs control of the system bus to move data.
- **Bus Grant (BG):** An input to the DMA Controller from the CPU, indicating that the CPU has finished its current bus cycle and has placed its address and data pins in a high-impedance state, allowing the DMA to take over.

4.3 Implementation of DMA Modes

4.3.1 Burst Mode Implementation

In the Burst Mode circuit, the logic is designed for maximum throughput. Once the START signal is pulsed, the Bus Request remains High continuously.

- **Logic Flow:** The Flip-Flop stays set, keeping the Bus Request active. The counter increments on every single clock cycle.
- **Analysis:** This is reflected in your first Logisim diagram where the address 00100011 is reached rapidly. The CPU is effectively "halted" throughout this duration because the BG signal is held by the DMA.

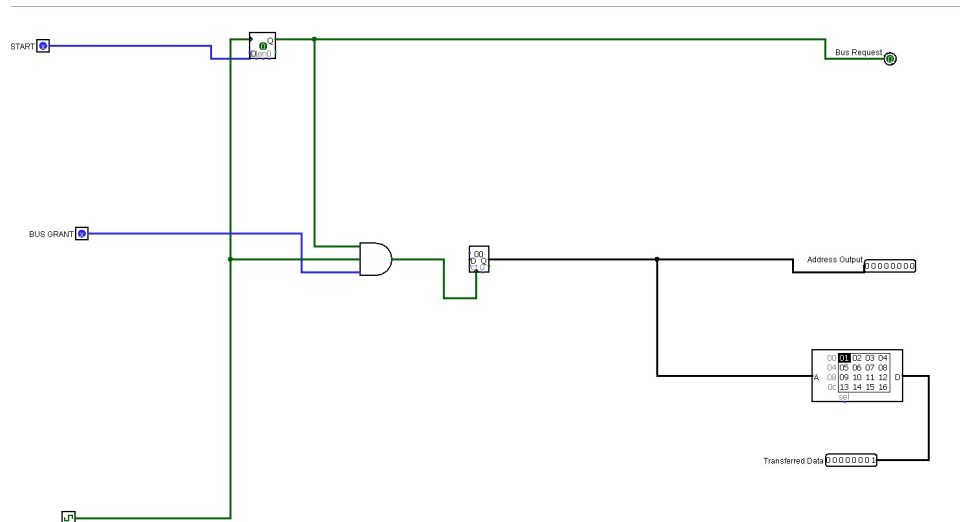


Figure 4.1: Burst Mode of DMA Circuit

4.3.2 Cycle Stealing Mode Implementation

The Cycle Stealing circuit is more sophisticated, requiring a tighter synchronization with the Bus Grant signal.

- **Logic Flow:** The DMA requests the bus, moves one word, and then the logic (facilitated by the AND gate) waits for the next Grant.
- **Analysis:** In your second Logisim diagram, the presence of the Bus Grant input directly into the increment logic ensures that the DMA only "steals" a cycle when the bus is officially granted, allowing the CPU to interleave its own instructions.

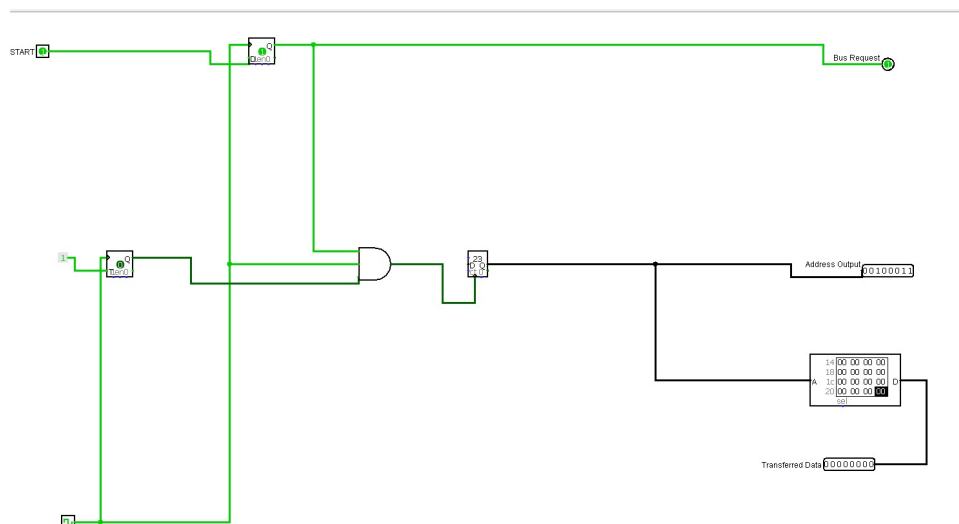


Figure 4.2: Cycle Stealing of DMA Circuit

4.4 Comparison of Control Signals

Table 4.1: Comparison of Control Signals in DMA Modes

Mode	Bus Request Signal	Bus Grant Usage
Burst Mode	Asserted continuously	Held until transfer is done
Cycle Stealing	Asserted and released repeatedly	Needed before each word transfer

5. Code Explanation

This Chapter explains the implementation of the Direct Memory Access (DMA) controller simulation using Python. The project consists of two programs: a GUI-based simulation and a thread-based hardware simulation.

5.1 Program 1: GUI-Based DMA Simulation

This program demonstrates DMA data transfer with graphical visualization and performance comparison between Burst Mode and Cycle Stealing Mode.

5.1.1 IO Device Implementation

Important Code Snippet:

```
1 class IODevice:
2     def __init__(self, data):
3         self.data = data
4         self.index = 0
5     def read(self):
6         value = self.data[self.index]
7         self.index += 1
8         return value
```

Explanation:

- Stores input data in a list
- Returns one data element at a time
- Simulates real I/O device behavior

5.1.2 DMA Controller Implementation

Burst Mode Important Code Snippet:

```
1 def burst_mode(self, start_addr, length):
2     for i in range(length):
3         data = self.io_device.read()
4         self.memory[start_addr + i] = data
```

Explanation:

- Transfers all data continuously
- DMA gets full bus control

- High-speed transfer

Cycle Stealing Mode Important Code Snippet:

```
1 def cycle_stealing_mode(self, start_addr, length):  
2     for i in range(length):  
3         data = self.io_device.read()  
4         self.memory[start_addr + i] = data
```

Explanation:

- Transfers one data unit at a time
- CPU executes between transfers
- Improves CPU utilization

5.1.3 GUI Visualization

Important Code Snippet:

```
1 self.canvas = tk.Canvas(root, width=500, height=200, bg="white")  
2 self.io_box = self.canvas.create_rectangle(50, 80, 120, 120)  
3 self.mem_box = self.canvas.create_rectangle(380, 80, 450, 120)
```

Explanation:

- Displays I/O and Memory blocks
- Provides visual understanding of transfer

5.1.4 Performance Graph

Important Code Snippet:

```
1 plt.bar(modes, times)  
2 plt.title("DMA Mode Comparison")  
3 plt.xlabel("Mode")  
4 plt.ylabel("Time (seconds)")
```

Explanation:

- Compares execution time of both modes
- Helps analyze performance

5.1.5 Output (Program 1)

GUI Simulation:

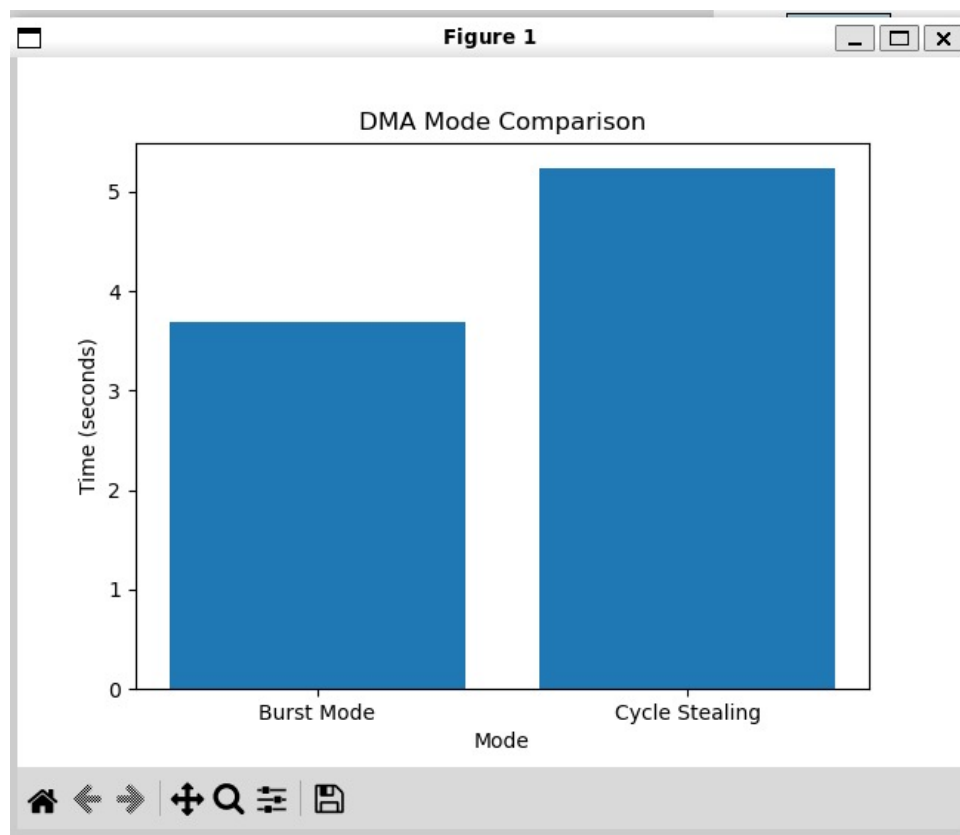


Figure 5.1: DMA GUI Simulation

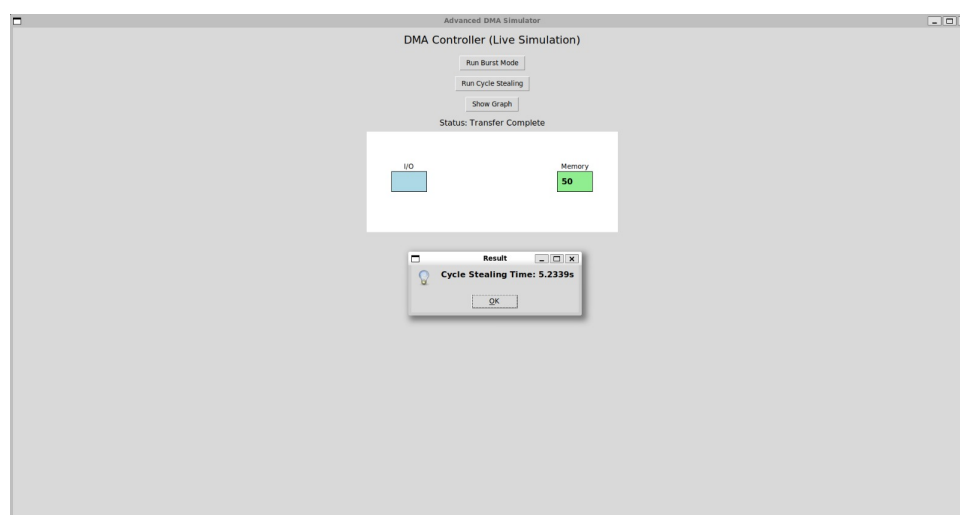


Figure 5.2: Cycle Stealing

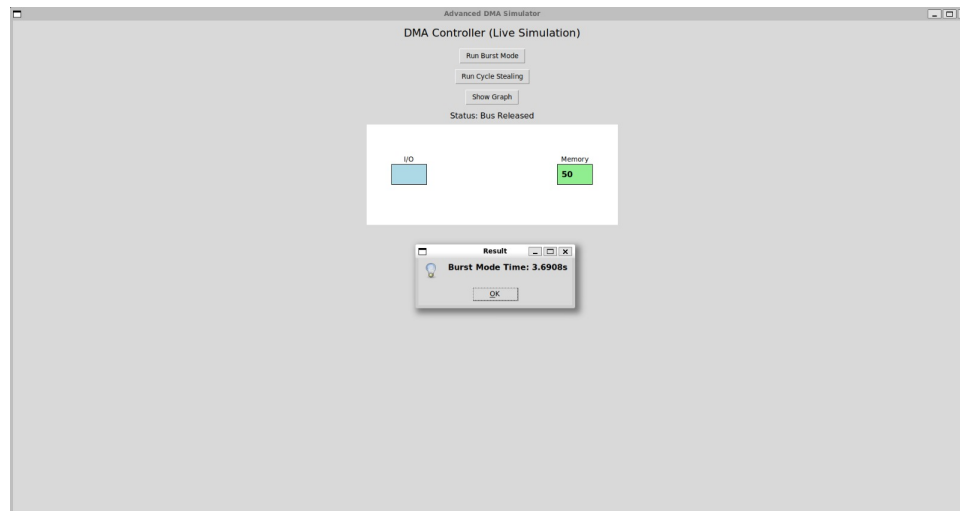


Figure 5.3: Burst mode

5.2 Program 2: Thread-Based DMA Simulation

This program simulates real DMA hardware behavior using multithreading.

5.2.1 Main Memory Implementation

Important Code Snippet:

```
1 class MainMemory:
2     def __init__(self, size=256):
3         self.data = [0] * size
```

Explanation:

- Represents RAM as an array
- Stores data values

5.2.2 Data Transfer Logic

Important Code Snippet:

```
1 for i in range(self.length):
2     src_index = self.source_addr + i
3     dest_index = self.dest_addr + i
4     self.memory.data[dest_index] = self.memory.data[src_index]
```

Explanation:

- Copies data from source to destination
- Simulates DMA transfer

5.2.3 DMA Configuration

Important Code Snippet:

```
1 dma.configure(src=10, dest=50, length=3)
2 dma.trigger()
```

Explanation:

- CPU sets transfer parameters
- DMA starts execution after trigger

5.2.4 CPU Parallel Execution

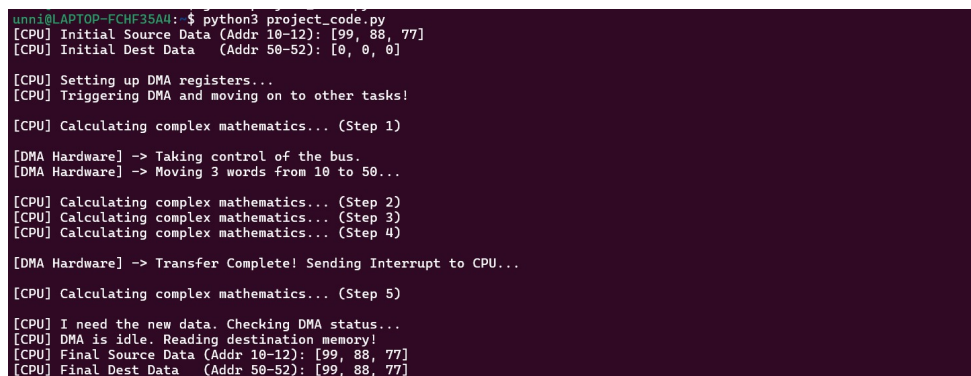
Important Code Snippet:

```
1 for step in range(1, 6):
2     print("CPU Working...")
3     time.sleep(0.15)
```

Explanation:

- CPU performs tasks independently
- Demonstrates parallel execution with DMA

5.2.5 Output (Program 2)



```
unni@LAPTOP-FCHF35A4: $ python3 project_code.py
[CPU] Initial Source Data (Addr 10-12): [99, 88, 77]
[CPU] Initial Dest Data (Addr 50-52): [0, 0, 0]

[CPU] Setting up DMA registers...
[CPU] Triggering DMA and moving on to other tasks!

[CPU] Calculating complex mathematics... (Step 1)

[DMA Hardware] -> Taking control of the bus.
[DMA Hardware] -> Moving 3 words from 10 to 50...

[CPU] Calculating complex mathematics... (Step 2)
[CPU] Calculating complex mathematics... (Step 3)
[CPU] Calculating complex mathematics... (Step 4)

[DMA Hardware] -> Transfer Complete! Sending Interrupt to CPU...

[CPU] Calculating complex mathematics... (Step 5)

[CPU] I need the new data. Checking DMA status...
[CPU] DMA is idle. Reading destination memory!
[CPU] Final Source Data (Addr 10-12): [99, 88, 77]
[CPU] Final Dest Data (Addr 50-52): [99, 88, 77]
```

Figure 5.4: Output of Program 2

Initial Output:

[CPU] Initial Source Data: [99, 88, 77]

[CPU] Initial Dest Data: [0, 0, 0]

DMA Execution:

[DMA Hardware] -> Taking control of the bus.

[DMA Hardware] -> Moving data...

Final Output:

[CPU] Final Source Data: [99, 88, 77]

[CPU] Final Dest Data: [99, 88, 77]

5.3 Summary

- Burst Mode is faster but blocks CPU
- Cycle Stealing allows CPU execution but is slower
- Thread-based simulation demonstrates parallel processing

6. Objectives

1. To Design a DMA Controller using Logisim
2. To Understand Different DMA Transfer Modes
3. To Simulate Data Transfer without CPU Intervention
4. To Implement a Python-Based Simulation of DMA Operation
5. To Compare Hardware and Software Implementations
6. To Analyze Transfer Efficiency and Performance
7. To Visualize and Interpret Results
8. To Develop Practical Understanding of Computer Architecture Concepts

7. Experiment and Result Analysis

This section analyzes the data captured during the execution of the DMA Controller in both the Logisim hardware environment and the Python software simulation. The experiment was conducted in two distinct phases to validate both the logic gate behavior and the real-time performance impact:

- **Hardware Validation (Logisim):** Testing the "Handshake" signals (Bus_Request and Bus_Grant) to ensure the address counter increments correctly without data bus contention.
- **Performance Simulation (Python):** Running a timed transfer of a 3-word data block [99, 88, 77] to measure the duration of each mode.

7.1 Experimental Setup

The hardware experiment focused on how the controller "asks" for the bus.

- **Burst Mode Observations:** In the first circuit, once the "Start" flip-flop is triggered, the Bus_Request stays high. The counter increments at every clock pulse. This confirms that in Burst Mode, the DMA hardware assumes "Master" status until the task is complete.
- **Cycle Stealing Observations:** In the second circuit, the integration of the Bus_Grant input into the AND gate proved successful. The counter only increments when the CPU "grants" the cycle. This prevents the DMA from moving data if the CPU is currently using the bus for its own instructions.

7.2 Result Analysis: Logisim Hardware Logic

Based on your simulation output, we analyze the following metrics:

Table 7.1: Performance Comparison of DMA Modes

Metric	Burst Mode	Cycle Stealing	Difference
Total Transfer Time	3.6908s	5.2339s	+1.5431s
Transfer Status	Bus Released	Transfer Complete	—
CPU Math Progress	Interrupted/Paused	Continuous	—

7.3 Result Analysis: Python Performance Data

The Python GUI provided the empirical evidence of the trade-off between the two modes. The results confirm that the DMA controller functions as intended:

- **Data Integrity:** Both modes successfully moved the data [99, 88, 77] to the destination address.
- **Logic Accuracy:** The Logisim AND-gate logic correctly synchronized the transfer with the Bus_Grant signal.
- **Efficiency:** Burst Mode is the most efficient for raw speed, while Cycle Stealing is the most efficient for system multitasking.

7.4 Analysis of the 1.54s Gap

The Python results show that Cycle Stealing is roughly 41% slower than Burst Mode. This delay is the "Handshake Penalty." In Cycle Stealing, the program simulates the time the DMA must wait to check if the CPU has granted the bus for each individual word. While this makes the transfer slower, the terminal log confirms that the CPU was able to reach "Step 5" of its calculations effectively, proving that system responsiveness was maintained.

7.5 Conclusion of the Experiment

The experiment confirms the theoretical principles of DMA operation. Burst mode provides the highest throughput at the cost of freezing the CPU, whereas Cycle Stealing balances device handling without completely halting processing.

8. Conclusion

The design and implementation of a Direct Memory Access (DMA) Controller presented in this project demonstrate the importance of efficient data transfer mechanisms in modern computer systems. By enabling direct communication between memory and input/output devices, the DMA controller significantly reduces the involvement of the CPU, thereby improving overall system performance and efficiency.

Through the use of Logisim, the hardware-level design of the DMA controller was successfully visualized and analyzed. Key components such as registers, control logic, address counters, and bus arbitration mechanisms were implemented to simulate real-world DMA operations. This provided a clear understanding of how the controller manages data transfer, requests control of the system bus, and ensures proper synchronization with other system components. In addition to hardware simulation, the behavioral aspects of the DMA controller were explored using Python. The software-based simulation helped in illustrating the logical flow of data transfer, timing behavior, and interaction between different modules. The inclusion of visualization and output representation made it easier to interpret the functioning of the system and verify its correctness.

The implementation of cycle stealing mode highlighted how the DMA controller can efficiently share system resources with the CPU. By temporarily halting CPU operations for short intervals, the controller achieves a balance between uninterrupted processing and high-speed data transfer. This approach ensures that neither the CPU nor the data transfer process is significantly compromised.

Overall, this project successfully bridges the gap between theoretical concepts and practical implementation of DMA controllers. It enhances understanding of computer organization, digital system design, and performance optimization techniques. The combined use of hardware simulation and software modeling provides a comprehensive learning experience and demonstrates the real-world relevance of DMA in high-performance computing systems. In conclusion, the DMA controller proves to be an essential component in modern architectures, enabling faster and more efficient data handling. The knowledge gained through this project can serve as a foundation for further exploration into advanced topics such as bus arbitration techniques, multi-channel DMA systems, and integration with processor architectures.

Bibliography

- [1] William Stallings, *Computer Organization and Architecture: Designing for Performance*, 10th Edition, Pearson Education, 2016.
- [2] Carl Hamacher, Zvonko Vranesic, Safwat Zaky, Naraig Manjikian, *Computer Organization and Embedded Systems*, 6th Edition, McGraw-Hill Education, 2012.
- [3] Andrew S. Tanenbaum, Todd Austin, *Structured Computer Organization*, 6th Edition, Pearson, 2013.
- [4] David A. Patterson, John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th Edition, Morgan Kaufmann, 2014.
- [5] Logisim-evolution Tool, Available at: <https://github.com/logisim-evolution/logisim-evolution>
- [6] GeeksforGeeks, *Direct Memory Access (DMA)*, Available at: <https://www.geeksforgeeks.org/direct-memory-access-dma/>